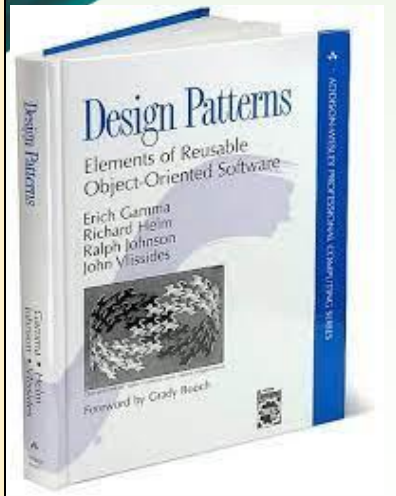
Stacks and Trees
1st part
Polish notation and its evaluation

Gregorics Tibor and Pintér Balázs

Patterns in programming

- Solving a programming problem is faster, and the resulting program is more secure, if we create the solution based on patterns that have proven successful in solving similar problems in the past.
- There are many such patterns in software engineering. In this course we are learning about:
  - algorithmic patterns
  - design patterns
- Design patterns support object-oriented modeling. We use them when designing class diagrams to make the model reusable, easy to modify, safe to operate, and efficient. Last but not least, they help us meet the SOLID design principles.



Exercise

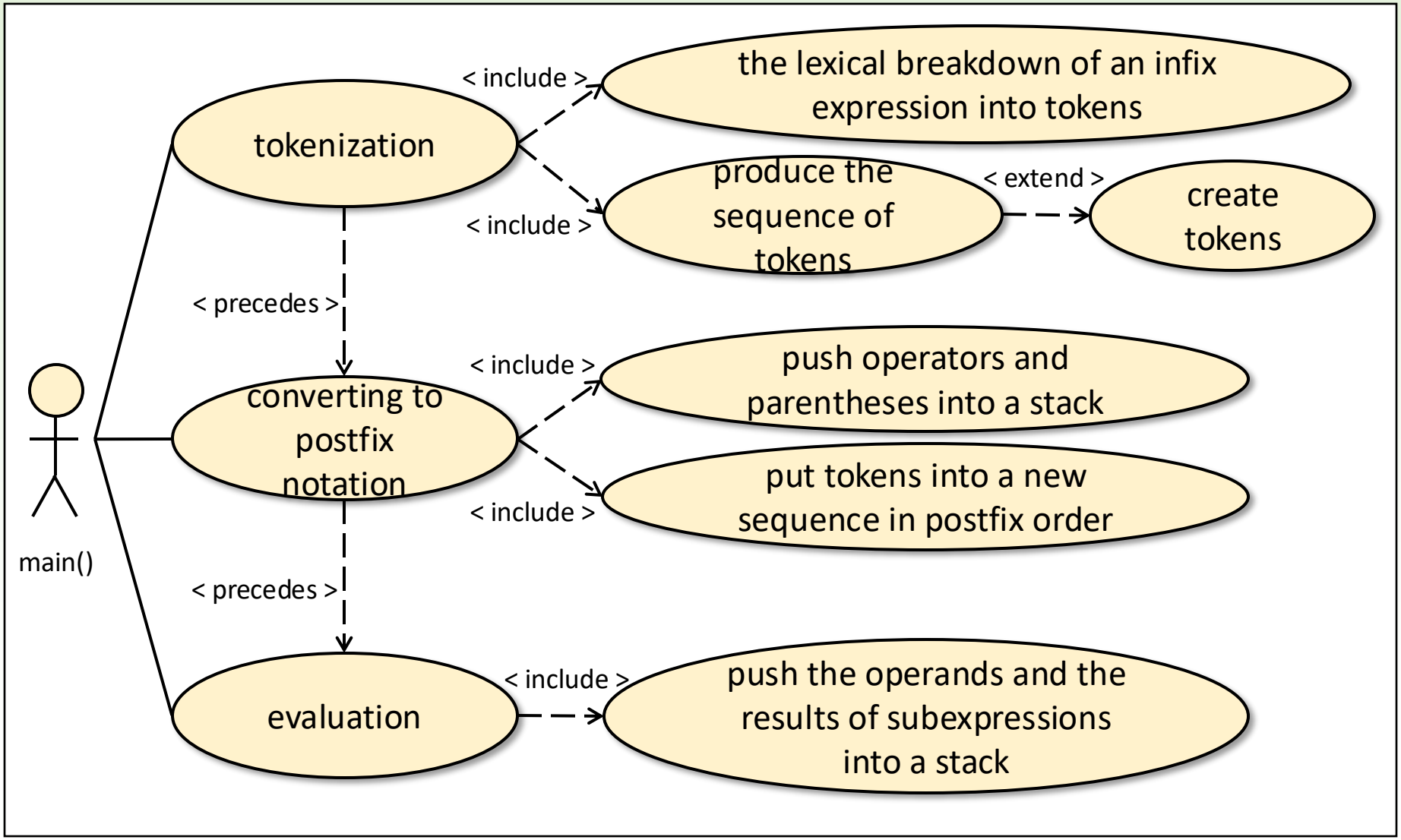
Let's convert an infix arithmetic expression to postfix form (RPN) and calculate its value. (The expression we are examining can only contain natural numbers.)

infix form		postfix form		value
5+(11+26)*(43-4)	→	**5 11 26 + 43 4 - * +**	→	**1448**

For both steps, a stack is typically used. During the transformation, the formula operators (operational symbols) and parentheses are stored in a stack, and during evaluation, operands and partial results, that is, numbers, are stored.

We need to be able to handle lexical units (tokens) separately in our expressions, that is, to handle operation symbols, parentheses, and numbers individually.

Analysis


main()
tokenization
converting to postfix notation
evaluation
< include >
the lexical breakdown of an infix expression into tokens
produce the sequence of tokens
< include >
< extend >
create tokens
< precedes >
< include >
push operators and parentheses into a stack
< include >
put tokens into a new sequence in postfix order
< precedes >
< include >
push the operands and the results of subexpressions into a stack

Objects

tokens: operators (Operator), operands(Operand),
parentheses (LeftP, RightP), all subclasses of the Token class

sequences: collections of tokens (their references) (List<Token>):
One stores the tokenized expression in infix form (infix),
the other stores it converted to postfix form.

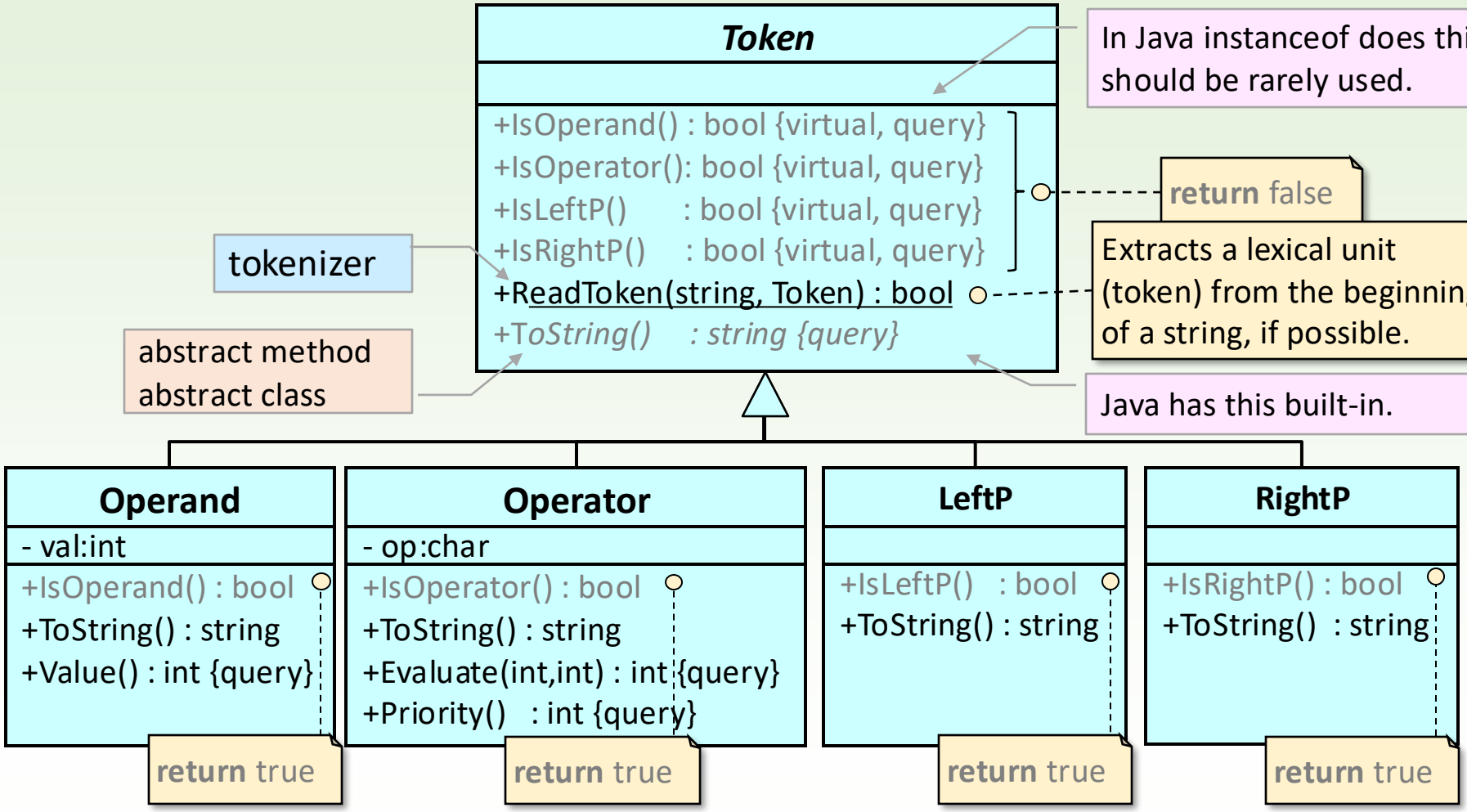
stacks: a stack (Stack<Token>) that stores tokens (their references) and a stack (Stack<int>) that stores integers,

string: the input infix expression (String)

Main program

```
public class PolishNotation {

    public static void main(String[] args) {
        try (Scanner scanner = new Scanner(System.in)) {
            char choice;

            do {
                System.out.println("\nGive me an arithmetic expression!\n");
                String input = scanner.nextLine().trim();

                try {
                    Expression exp = new Expression(input);
                    int result = exp.evaluate();

                    System.out.println("value: " + result);

                } catch (ExpressionProcessingException e) {
                    System.out.println(e.getMessage());
                }

                System.out.print("\nDo you continue? Y/N ");
                choice = scanner.next().toLowerCase().charAt(0);
                scanner.nextLine(); // consume newline

            } while (choice != 'n');
        }
    }
}
```

Token classes (first attempt)



Factory function of Token class

```
public class TokenReader {
    static final String digits = "0123456789";
    public static Token readToken(StringBuilder input) {
        if (input.length() == 0) return null;
        int i = 1;
        Token token;
        switch (input.charAt(0)) {
            case '+': token = new Operator(/* ... */); break;
            case '-': token = new Operator(/* ... */); break;
            case '*': token = new Operator(/* ... */); break;
            case '/': token = new Operator(/* ... */); break;
            case '(': token = new LeftP(); break;
            case ')': token = new RightP(); break;
            case '0': case '1': case '2': case '3': case '4':
            case '5': case '6': case '7': case '8': case '9':
                String number = "";
                for (i = 0; i < input.length() && digits.contains("" + input.charAt(i)); ++i)
{
                    number += input.charAt(i);
                }
                token = new Operand(Integer.parseInt(number));
                break;
            default:
                throw new IllegalElementException();
        }
        input.delete(0, i);
        return token;
    }
}
```

input parameter

concatenation
until condition is met

truncates tokenized characters from the beginning of the input

Operator class (first attempt)

```
public class Operator extends Token {
    private final char op;
    public Operator(char o) { this.op = o; }
    @Override public String toString() { return Character.toString(op); }
    public int evaluate(int leftValue, int rightValue) {
        switch (op) {
            case '+': return leftValue + rightValue;
            case '-': return leftValue - rightValue;
            case '*': return leftValue * rightVal
            case '/': return leftValue / rightVal
            default: return 0;
        }
    }
    public int priority() {
        switch (op) {
            case '+':
            case '-': return 1;
            case '*':
            case '/': return 2;
            default: return 3;
        }
    }
}
```

Single responsibility

Open-Closed

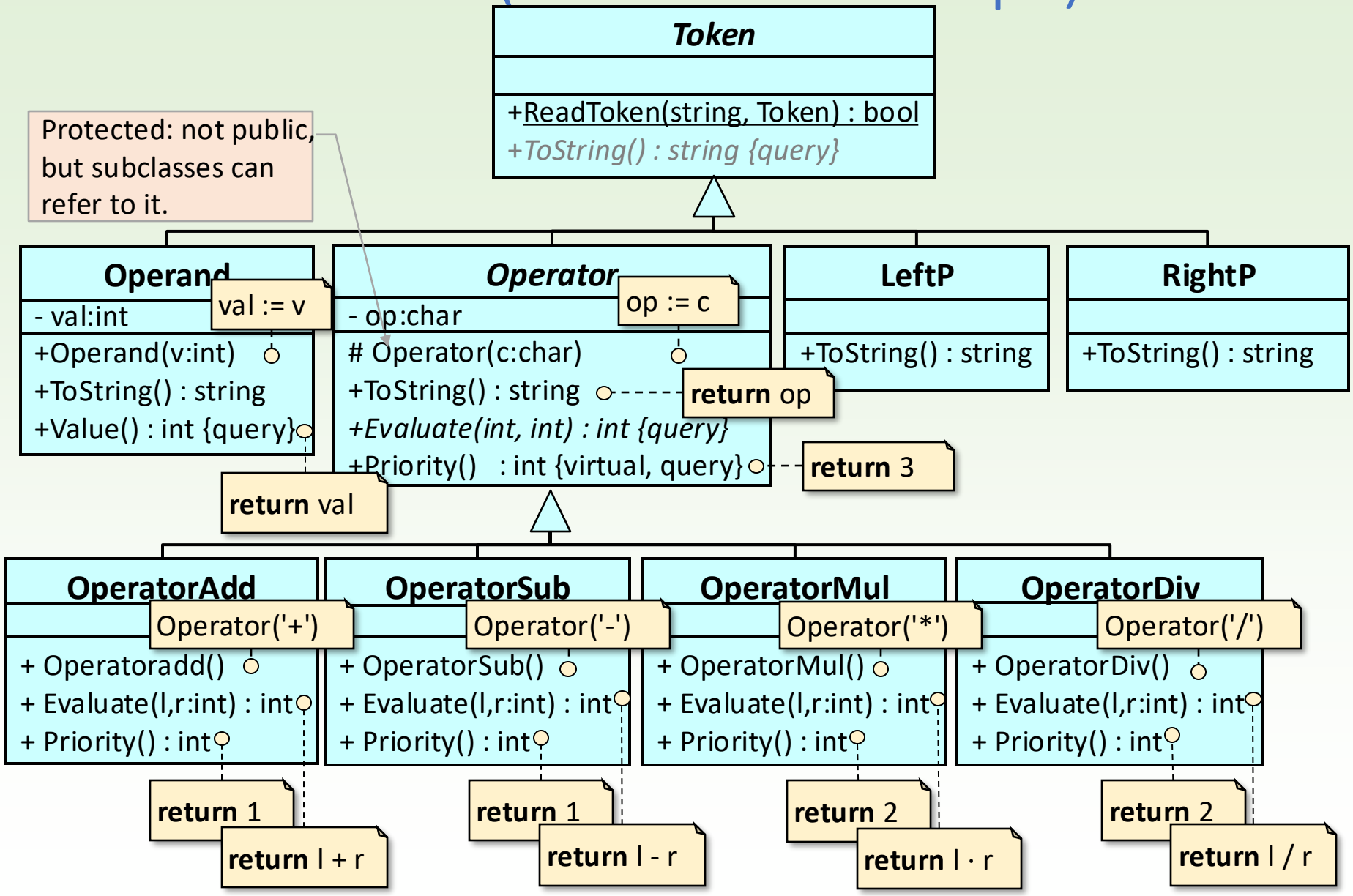
Liskov's substitution

I

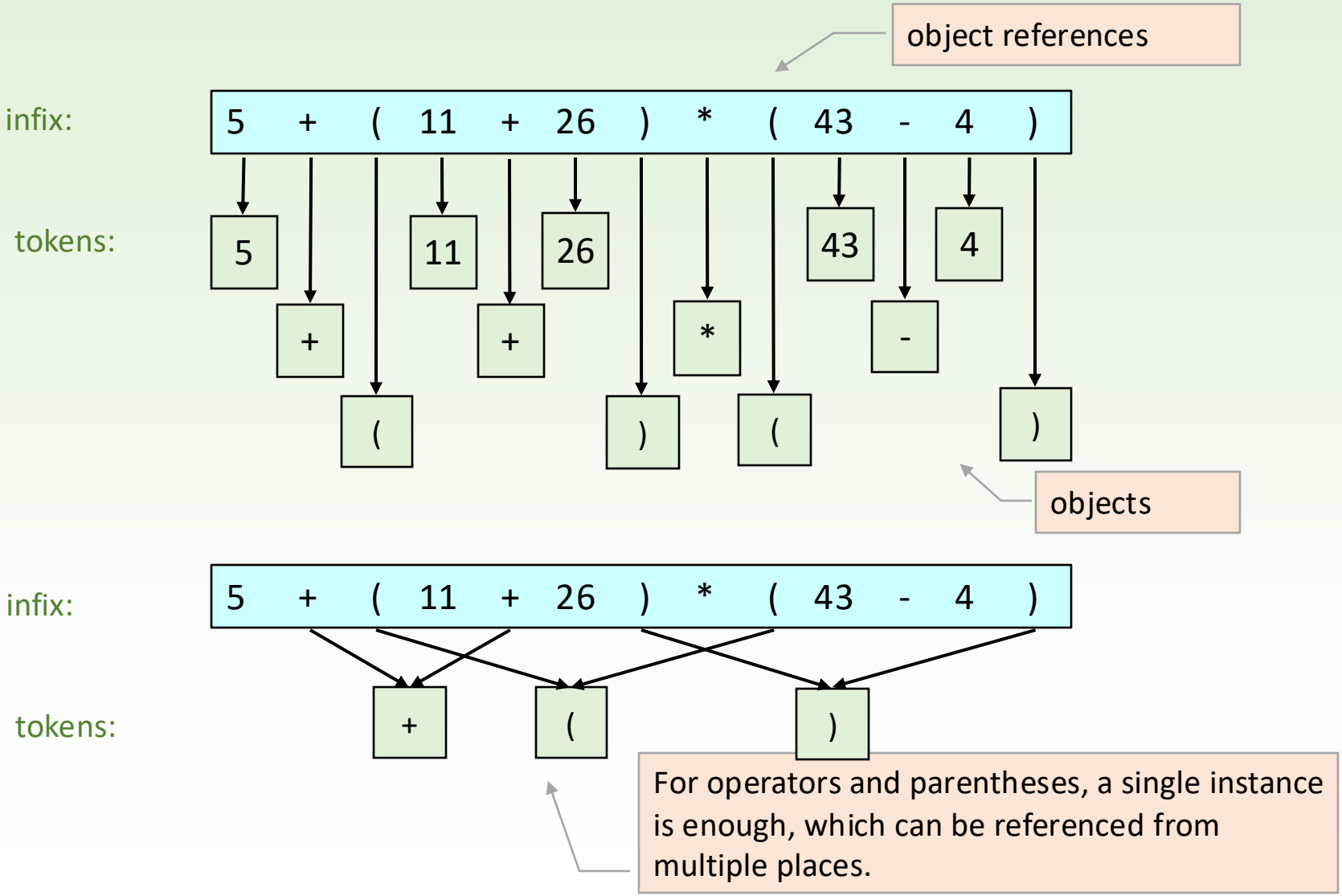
D

This code violates the Open/Closed Principle:
If we introduce more operation types, we'd have to modify the code of multiple methods.

Token classes (second attempt)

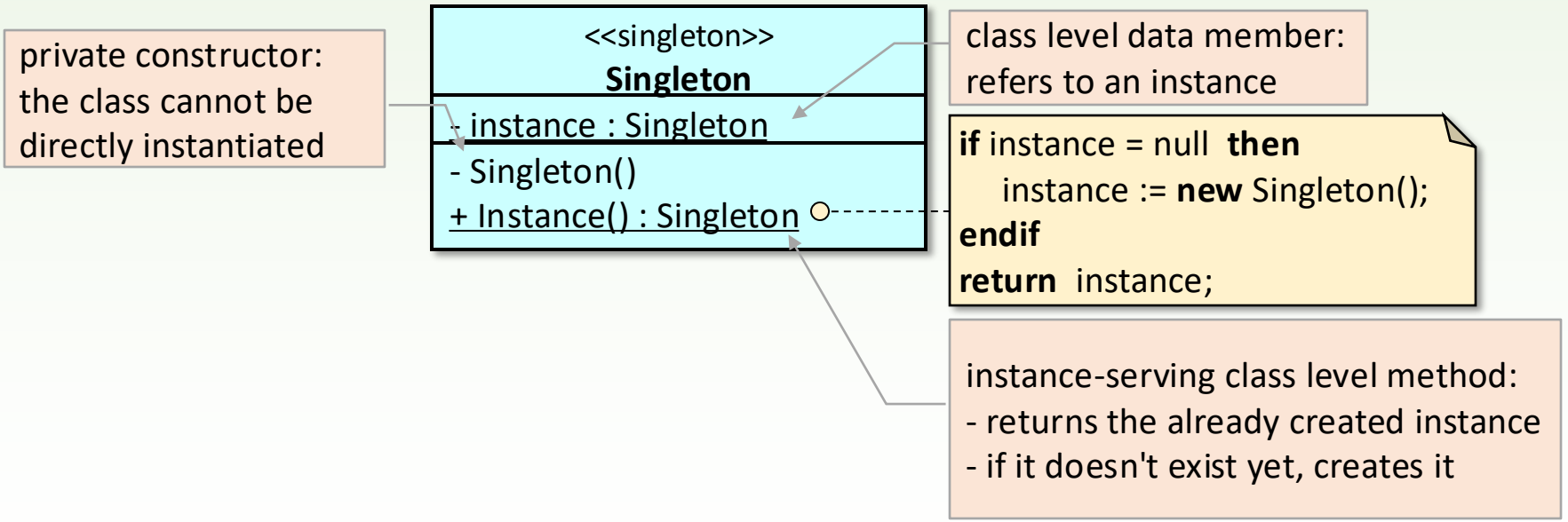


Memory footprint

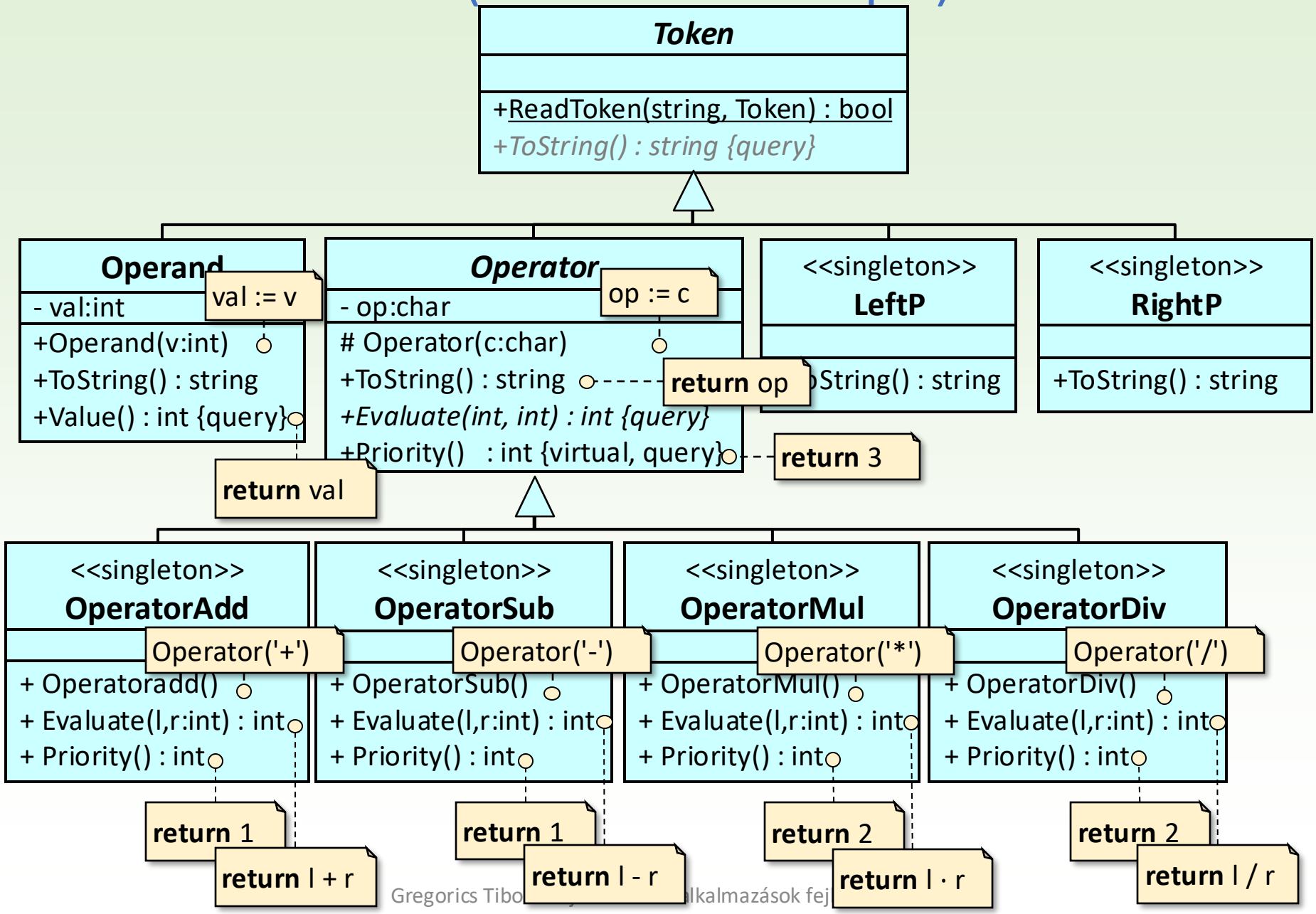


Singleton Design Pattern

- When a class needs to instantiate at most one object regardless of the number of instantiation requests.



Token classes (third attempt)



LeftP and RightP class as Singletons

```
public class RightP extends Token {
    private static RightP instance;

    private RightP() {}
```

```
public class LeftP extends Token {
    private static LeftP instance;

    private LeftP() {}

    public static LeftP getInstance() {
        if (instance == null) {
            instance = new LeftP();
        }
        return instance;
    }


    @Override
    public String toString() {
        return "(";
    }

}
```

refers to the already created single instance, or null

cannot be called from outside the class

this factory function should be called instead of the constructor

```
if (instance == null) instance = new LeftP()
```

Operator and its singleton subclasses

```
public abstract class Operator extends Token {
    private final char op;
    protected Operator(char o) { this.op = o; }
    public abstract int evaluate(int leftValue, int rightValue);
    public int priority() { return 3; }
    @Override
    public String toString() { return String.valueOf(op); }
}
```

```
public class OperatorAdd extends Operator {
```

```
public class OperatorSub extends Operator {
```

```
public class OperatorMul extends Operator {
```

```
public class OperatorDiv extends Operator {
    private static OperatorDiv instance;

    private OperatorDiv() { super('/'); }

    @Override
    public int evaluate(int leftValue, int rightValue)
    { return leftValue / rightValue; }

    @Override
    public int priority() { return 2; }

    public static OperatorDiv getInstance() {
        if (instance == null) {
            instance = new OperatorDiv();
        }
        return instance;
    }
}
```

calling the constructor of the parent class

the branches disappeared

Tokenization - again

```
public static TokenResult readToken(String input) throws IllegalElementException {
    if (input.isEmpty()) {
        return new TokenResult(null, input);
    }
    int i = 1;
    final String digits = "0123456789";

    char firstChar = input.charAt(0);
    Token token = switch (firstChar) {
        case '+' -> OperatorAdd.getInstance();
        case '-' -> OperatorSub.getInstance();
        case '*' -> OperatorMul.getInstance();
        case '/' -> OperatorDiv.getInstance();
        case '(' -> LeftP.getInstance();
        case ')' -> RightP.getInstance();
        case '0', '1', '2', '3', '4', '5', '6', '7', '8', '9' -> {
            StringBuilder str = new StringBuilder();
            for (i = 0; i < input.length() && digits.indexOf(input.charAt(i)) >= 0; ++i) {
                str.append(input.charAt(i));
            }
            yield new Operand(Integer.parseInt(str.toString()));
        }
        default -> throw new IllegalElementException(firstChar);
    };
    String remainingInput = input.substring(i);
    return new TokenResult(token, remainingInput);
}
```

```
public static class IllegalElementException extends Exception {
    private final char ch;
    public IllegalElementException(char c)
    {   super("Illegal character: " + c);  this.ch = c; }
    public String what() { return String.valueOf(ch);  }
}
```

The exception object can also have data members and methods.

Tokenization - again – continued

```
public static class TokenResult {
    private final Token token;
    private final String remainingInput;

    public TokenResult(Token token, String remainingInput) {
        this.token = token;
        this.remainingInput = remainingInput;
    }

    public Token getToken() {
        return token;
    }

    public String getRemainingInput() {
        return remainingInput;
    }
}
```

```
public static class IllegalElementException extends Exception {
    private final char ch;
    public IllegalElementException(char c)
    {   super("Illegal character: " + c);  this.ch = c; }
    public String what() { return String.valueOf(ch);  }
}
```

The exception object can also have data members and methods.

Expression: evaluation steps

```
public class Expression {
    private String input;
    private List<Token> infixTokens;
    private List<Token> postfixTokens;
    public Expression(String input) {
        this.input = input;
    }

    public int evaluate() throws ExpressionProcessingException {
        if (infixTokens == null) {
            tokenizeInput();
        }
        if (postfixTokens == null) {
            convertToPostfix();
        }
        return evaluateExpression();
    }

    public void tokenizeInput() throws ExpressionProcessingException { … }

    public void convertToPostfix() throws ExpressionProcessingException { … }

    public int evaluateExpression() throws ExpressionProcessingException { … }
```

Expression: tokenization

```
public void tokenizeInput() throws ExpressionProcessingException {

    this.infixTokens = new ArrayList<>();

    try {
        while (!input.isEmpty()) {
            Token.TokenResult result = Token.readToken(input);
            Token token = result.getToken();
            if (token != null) {
                infixTokens.add(token);
            }
            input = result.getRemainingInput();
        }
    } catch (Token.IllegalElementException ex) {
        throw new ExpressionProcessingException("Illegal character: " +
ex.what());
    }

    if (infixTokens.isEmpty()) {
        throw new ExpressionProcessingException("Empty expression");
    }
}
```

reads token

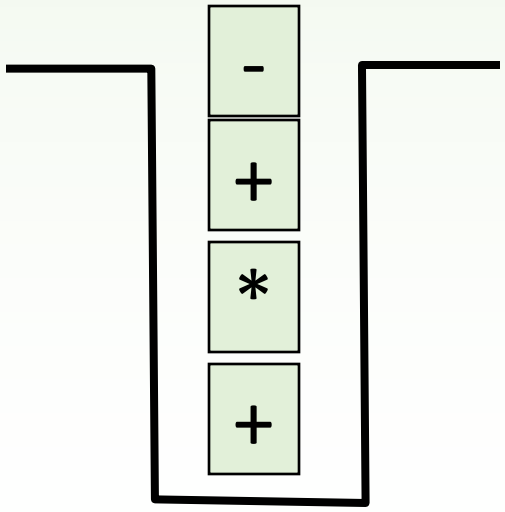
the reading roll

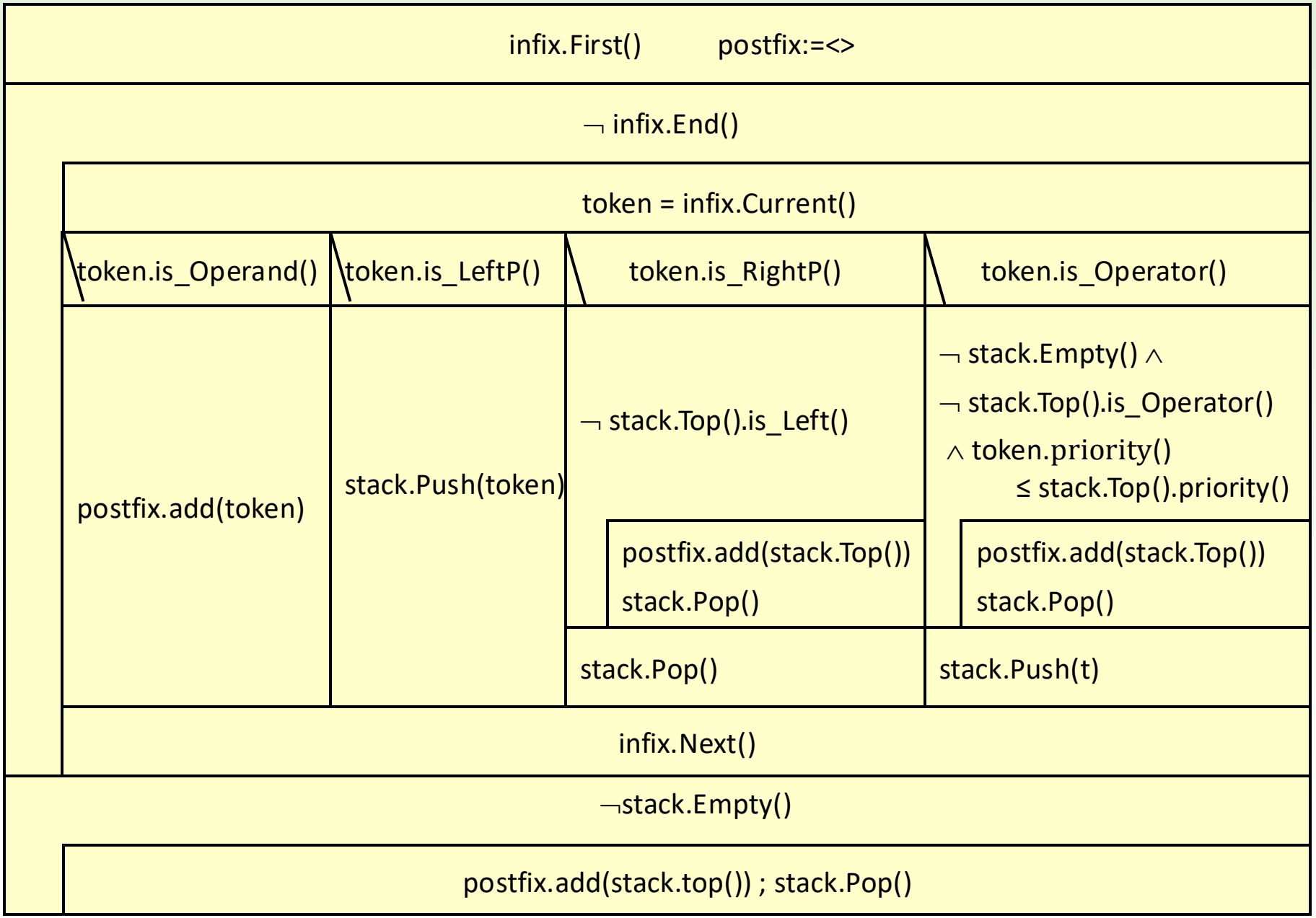
no further action is taken

Converting from infix to postfix

We place the opening parentheses and operation symbols of the input sequence into a stack (the operation with lower precedence always swaps places with the higher precedence one below it). All other symbols are copied directly to the output sequence. When reading a closing parenthesis or at the end of processing the input sequence, we empty the stack up to the topmost opening parenthesis into the output sequence.

5	+	(	11	+	26	)	*	(	43	-	4	)




infix.First()
postfix:=<>
¬ infix.End()
token = infix.Current()
token.is_Operand()
token.is_LeftP()
token.is_RightP()
token.is_Operator()
postfix.add(token)
stack.Push(token)
¬ stack.Top().is_Left()
¬ stack.Empty() ∧
¬ stack.Top().is_Operator()
∧ token.priority()
≤ stack.Top().priority()
postfix.add(stack.Top())
stack.Pop()
postfix.add(stack.Top())
stack.Pop()
stack.Pop()
stack.Push(t)
infix.Next()
¬stack.Empty()
postfix.add(stack.top()) ; stack.Pop()

Expression: Conversion to postfix form

```
public void convertToPostfix() throws ExpressionProcessingException {
  Stack<Token> operatorStack = new Stack<>();
  this.postfixTokens = new ArrayList<>();
  for (Token token : infixTokens) {
    switch (token) {
        case Operand operand ->
            postfixTokens.add(operand);
        case LeftP leftP ->
            operatorStack.push(leftP);
        case RightP rightP -> {
            try {
                while (!(operatorStack.peek() instanceof LeftP))
                    postfixTokens.add(operatorStack.pop());
                }
                operatorStack.pop(); // Discard the left parenthesis
            } catch (Exception e) {
                throw new ExpressionProcessingException("Syntax error: less left
parenthesis than right ones");
            }
        }
        ……..
```

the four-pronged fork on the next slide

error possibility:
more opening parentheses than closing ones

Expression: Conversion to postfix form

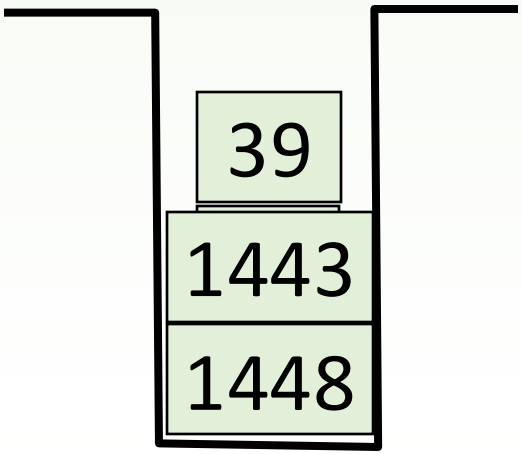
```
        ……..
         case Operator currentOperator -> {
            while (!operatorStack.empty() && operatorStack.peek() instanceof Operator
                    && currentOperator.priority() <= ((Operator) operatorStack.peek()).priority()) {
                postfixTokens.add(operatorStack.pop());
            }
            operatorStack.push(token);
        }
        default -> {
            throw new ExpressionProcessingException("Syntax error: unrecognized token");
        }
    }
  }
  // Pop any remaining operators from the stack
  while (!operatorStack.empty()) {
      if (operatorStack.peek() instanceof LeftP) {
          throw new ExpressionProcessingException("Syntax error: more left parentheses than right ones");
      } else {
          postfixTokens.add(operatorStack.pop());
      }
  }
}
```

error possibility: more closing parentheses than opening ones

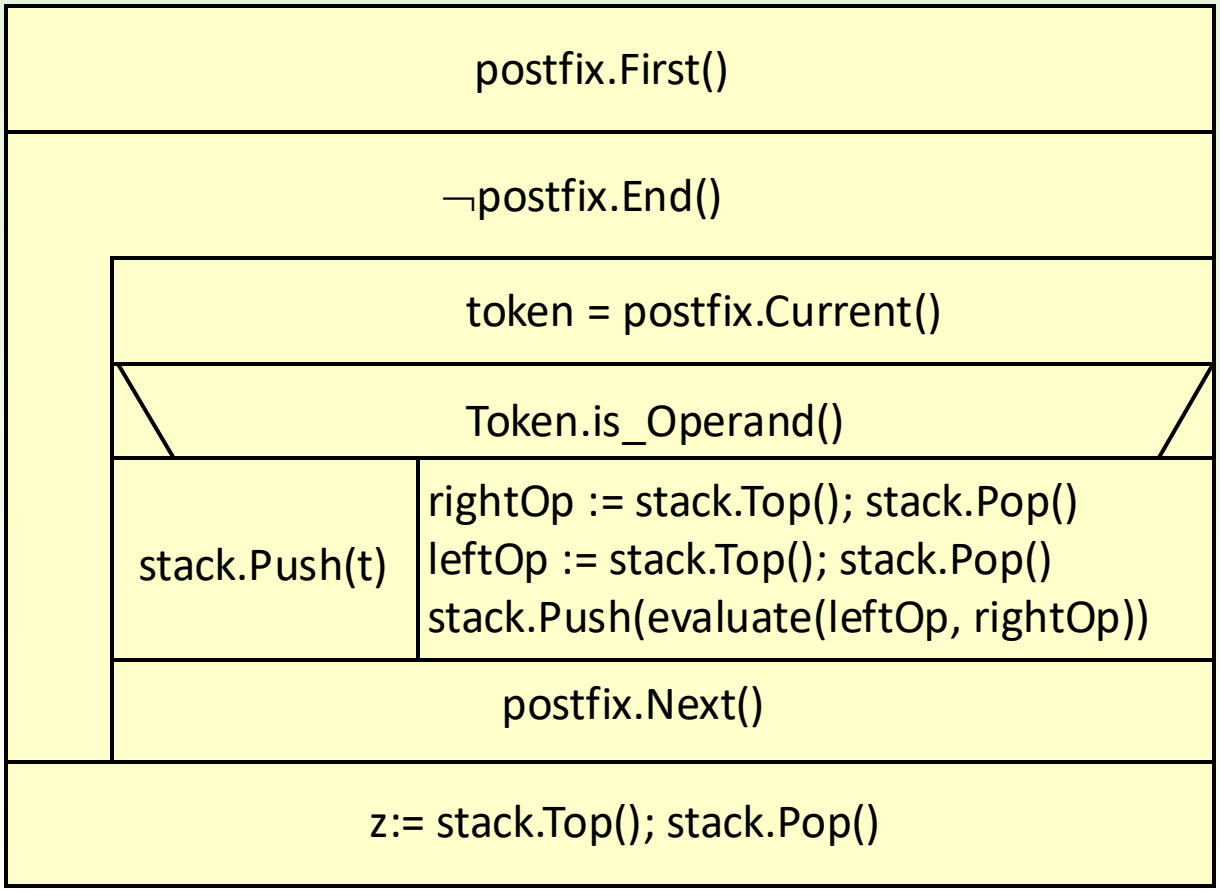
Evaluation of postfix form

We read the operands of the postfix form (in their order in the input) and place them in a stack. When reading an operation symbol, we take the two values at the top of the stack (we only have binary operations), process them with the operation, and put the result back on the stack. At the end of processing, we find the value of the expression in the stack.

5	11	26	+	43	4	-	*	+



Evaluation algorithm



Evaluation

```
public int evaluateExpression() throws ExpressionProcessingException {
  Stack<Integer> valueStack = new Stack<>();
  for (Token token : postfixTokens) {
    switch (token) {
       case Operand operand ->
         valueStack.push(operand.value());
       case Operator operator -> {
          if (valueStack.size() < 2) {
            throw new ExpressionProcessingException("Not enough operands");
          }
          int rightOperand = valueStack.pop();
          int leftOperand = valueStack.pop();
          valueStack.push(operator.evaluate(leftOperand, rightOperand));
        }
        default ->
          throw new ExpressionProcessingException("Unknown token type");
    }
  }
  if (valueStack.size() != 1) {
    throw new ExpressionProcessingException("Syntax error: more operands than
operators");
  }
  return valueStack.pop();
}
```

list

operand refers to a token cast to type Operand

error possibility: few operands

error possibility: multiple operands

Stacks and Trees
2nd part
Binary tree and its traversal

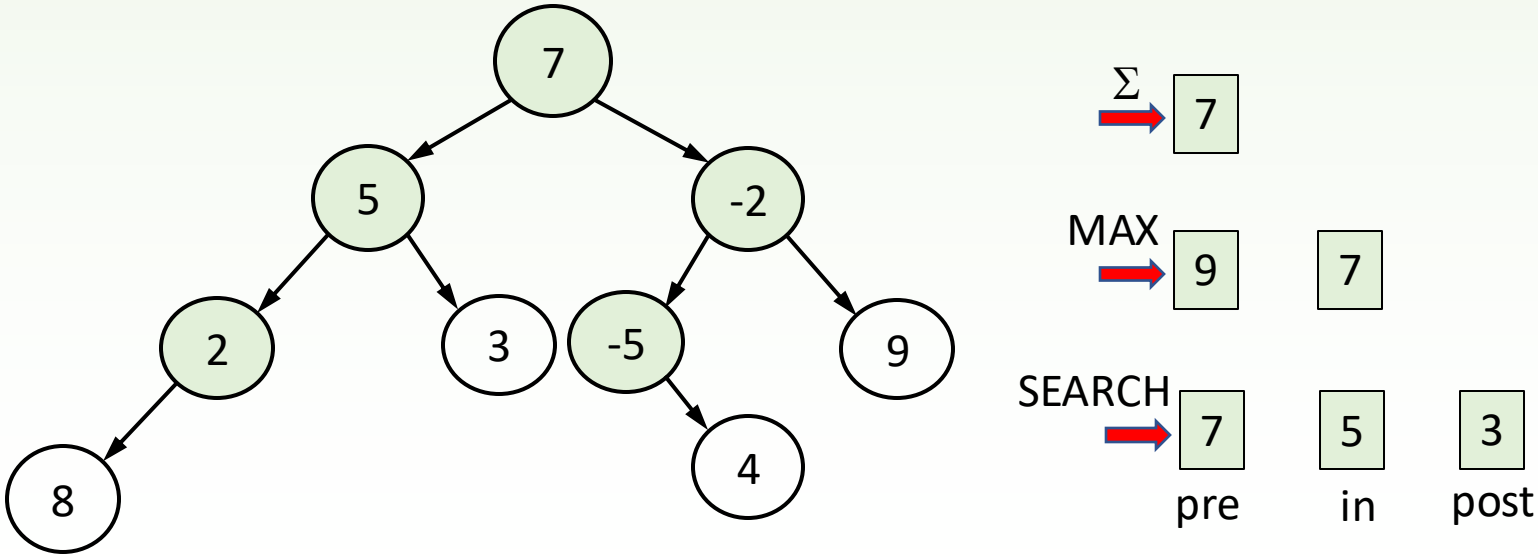
Gregorics Tibor and Pintér Balázs

Task: Traversal of a binary tree

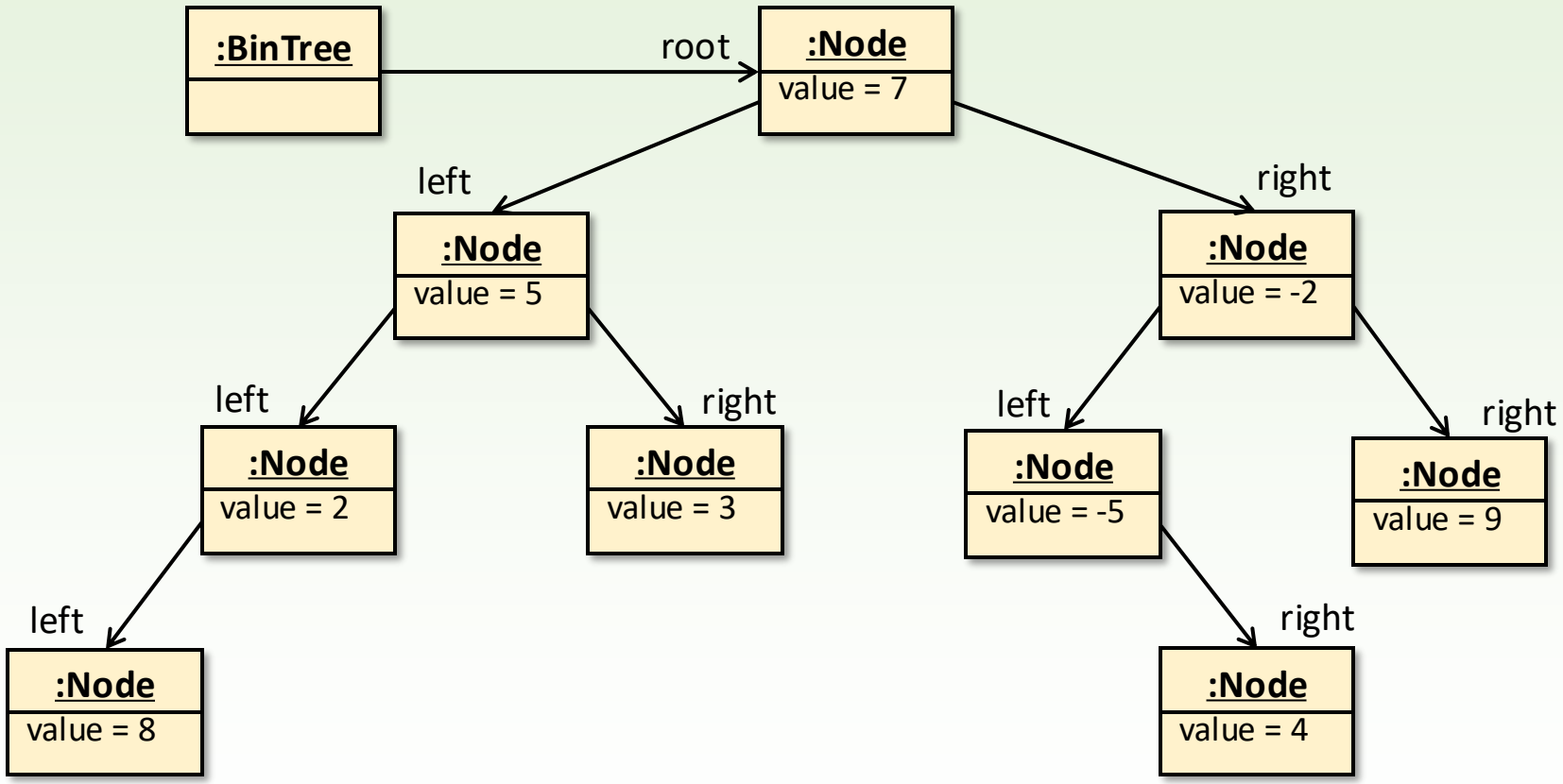
Let's build a directed binary tree from given integers such that each value is placed in a different node, but which node it goes into is determined randomly.

Let's print the values stored in the nodes using various traversal strategies.

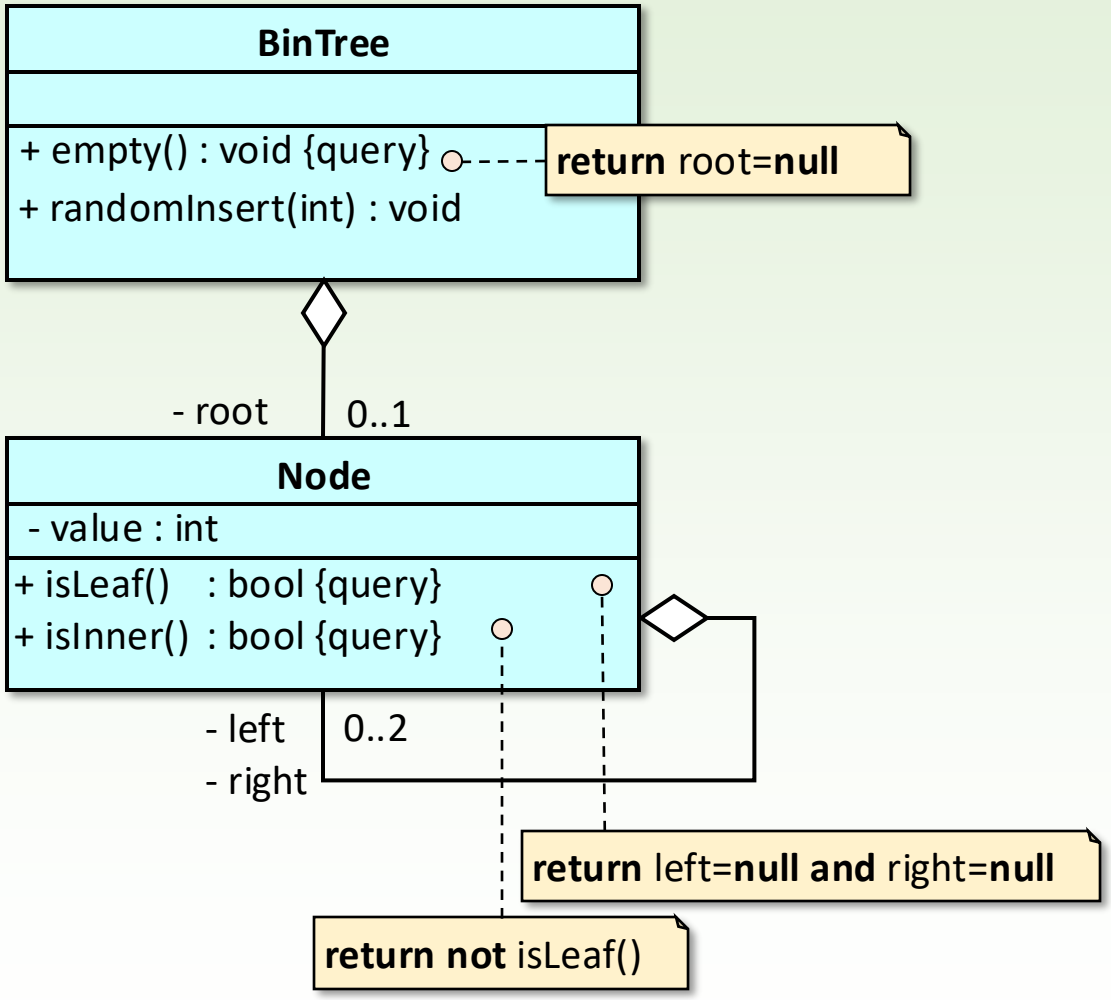
Let's perform the summation of the values of the internal (non-leaf) nodes of a tree, find the maximum among the values of all nodes or just the internal nodes, as well as find the "first" odd number according a traversal strategy.



Object diagram of binary tree



Class diagram

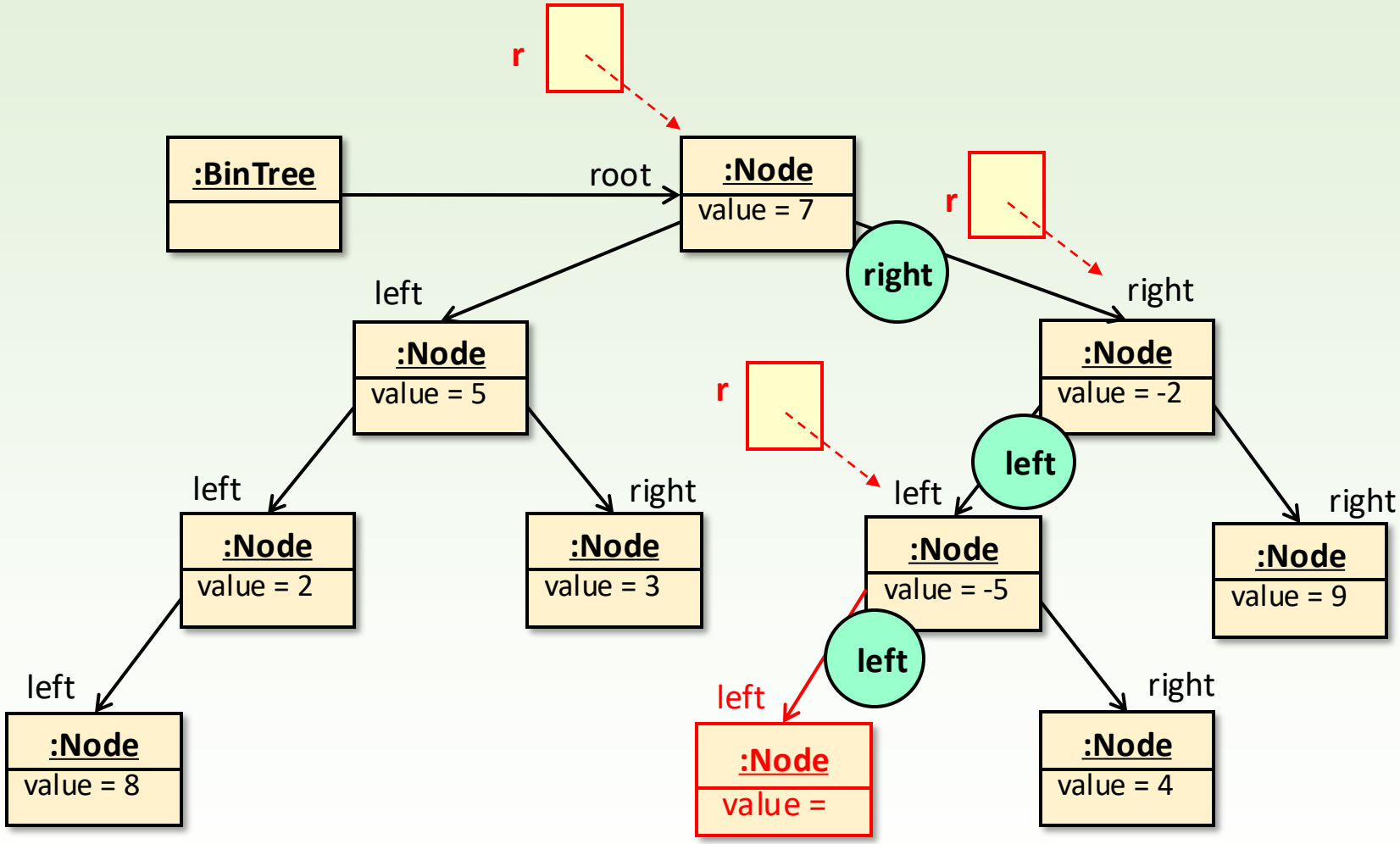


```
public class Node
{
    private int value;
    private Node left;
    private Node right;

    public Node(int v)
    {
        Value = v;
        Left = null;
        Right = null;
    }
    public int getValue() { return value; }
    public Node getLeft() { return left; }
    public void setLeft(Node left) { this.left = left; }
    public Node getRight() { return right; }
    public void setRight(Node right) { this.right = right; }

    public boolean isLeaf() { return left == null && right == null; }
    public boolean isInternal() { return !isLeaf(); }
}
```

```
public class BinTree {
    public static class NoRootException extends Exception {
        public NoRootException() {
            super();
        }
    }

    private Node root;
    private final Random rand;

    public int getRoot() throws NoRootException {
        if (root == null) throw new NoRootException();
        return root.getValue();
    }

    public boolean isEmpty() {
        return root == null;
    }

    public BinTree() {
        root = null;
        rand = new Random(new Date().getTime());
    }

    public void randomInsert(int e) { … }
}
```

Initializing the random number generator

Inserting a new node into the tree



Inserting a new node into the tree

```
public void randomInsert(int e) {
  if (root == null) {
    root = new Node(e);
  } else {
    Node r = root;
    boolean l = rand.nextInt(2) != 0;
    while (l ? r.getLeft() != null : r.getRight() != null) {
      if (l) r = r.getLeft();
      else r = r.getRight();
      l = rand.nextInt(2) != 0;
    }

    if (l) r.setLeft(new Node(e));
    else r.setRight(new Node(e));
  }
}
```

conditional expression

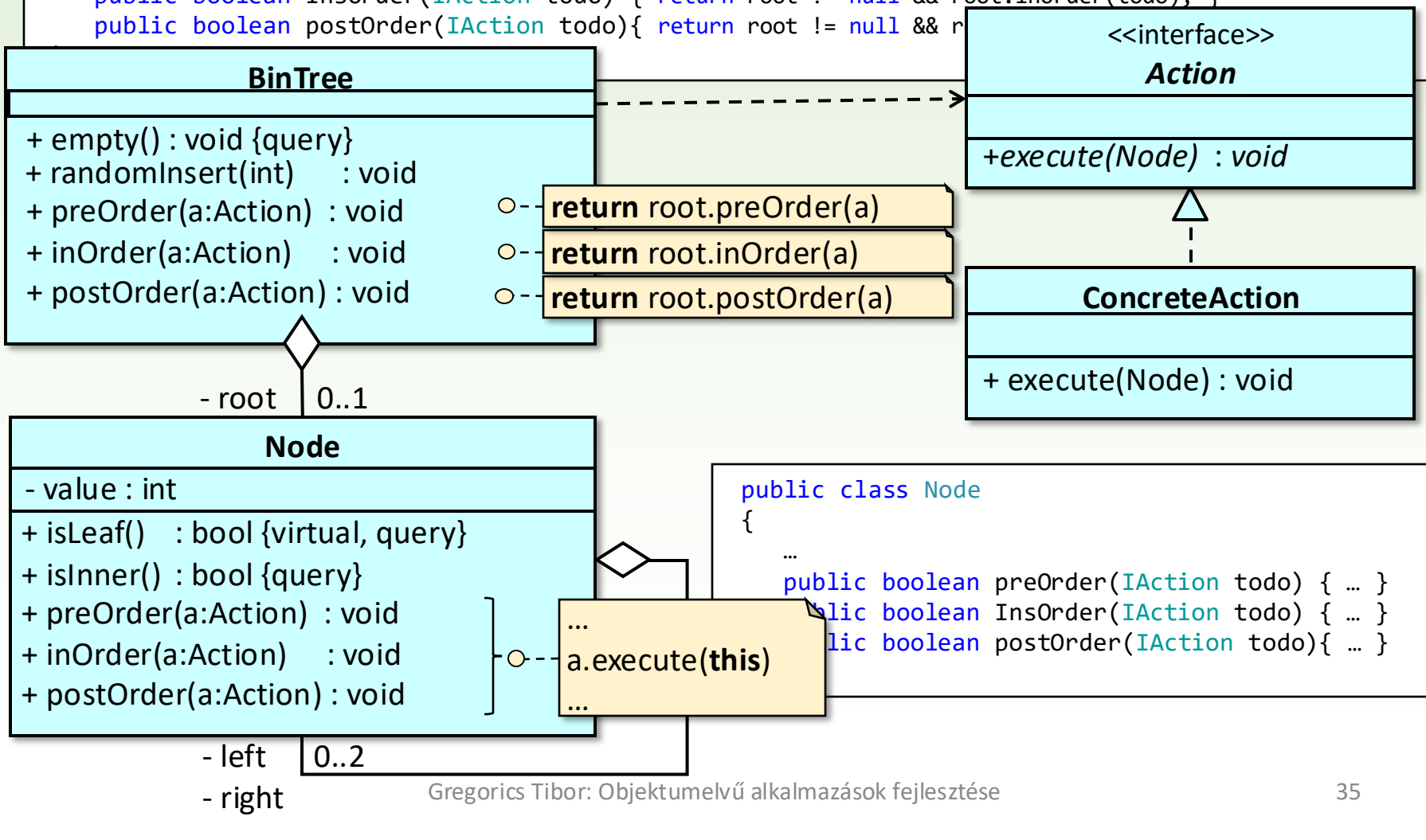
```
BinTree t = new BinTree();
Scanner scanner = new Scanner(System.in);

System.out.println("Give the elements of the tree, one on each line, end with
0: ");
int i;
while ((i = Integer.parseInt(scanner.nextLine())) != 0) {
    t.randomInsert(i);
}
```

tree structure in the main program

Class Diagram

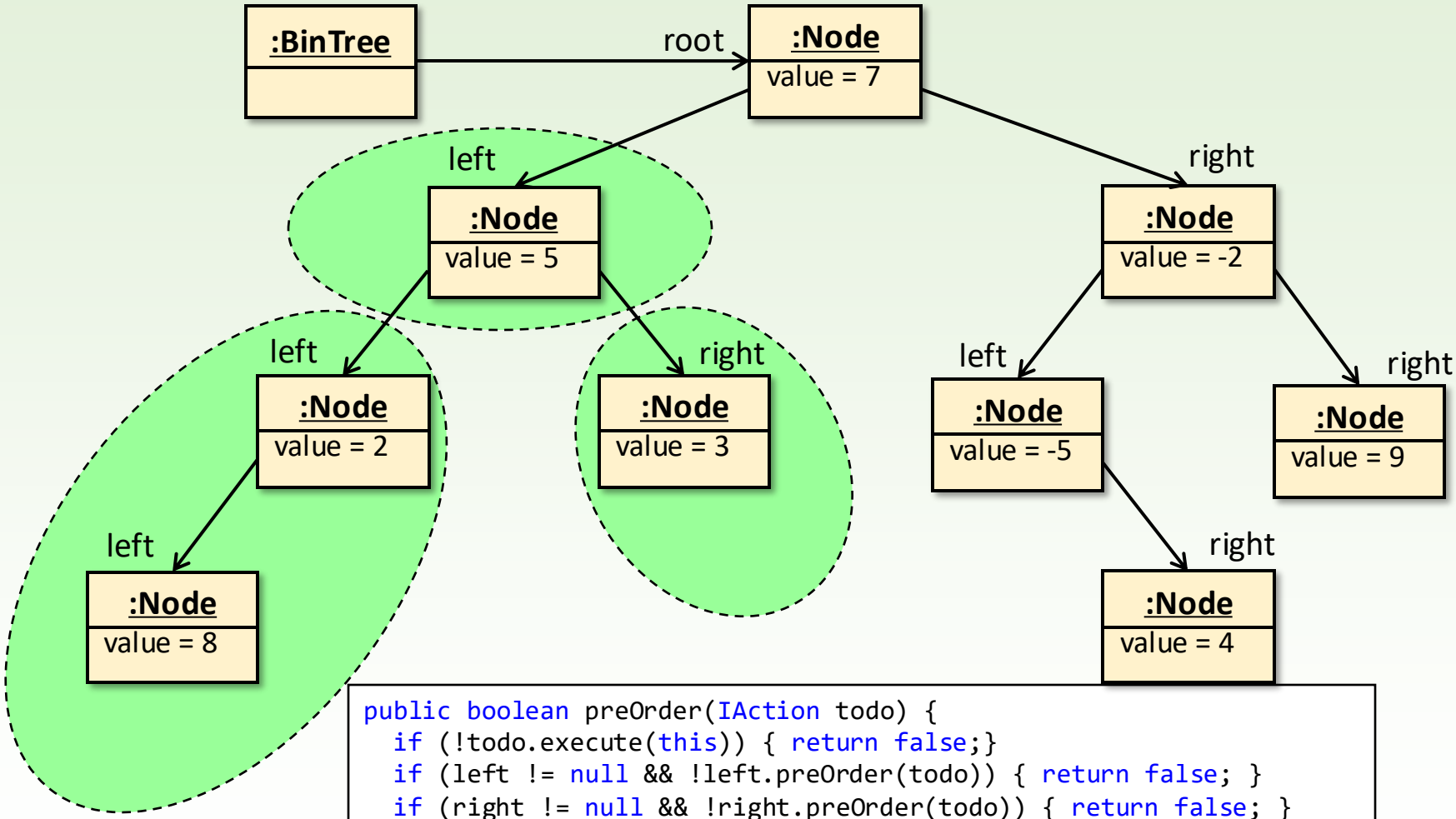
```
public interface IAction
{
    boolean execute(Node node);
}
```

```
public class BinTree
{
    …
    public boolean preOrder(IAction todo) { return root != null && root.preOrder(todo); }
    public boolean InsOrder(IAction todo) { return root != null && root.inOrder(todo); }
    public boolean postOrder(IAction todo){ return root != null && r
```




```
public class Node
{
    …
    public boolean preOrder(IAction todo) { … }
    public boolean InsOrder(IAction todo) { … }
    public boolean postOrder(IAction todo){ … }
```

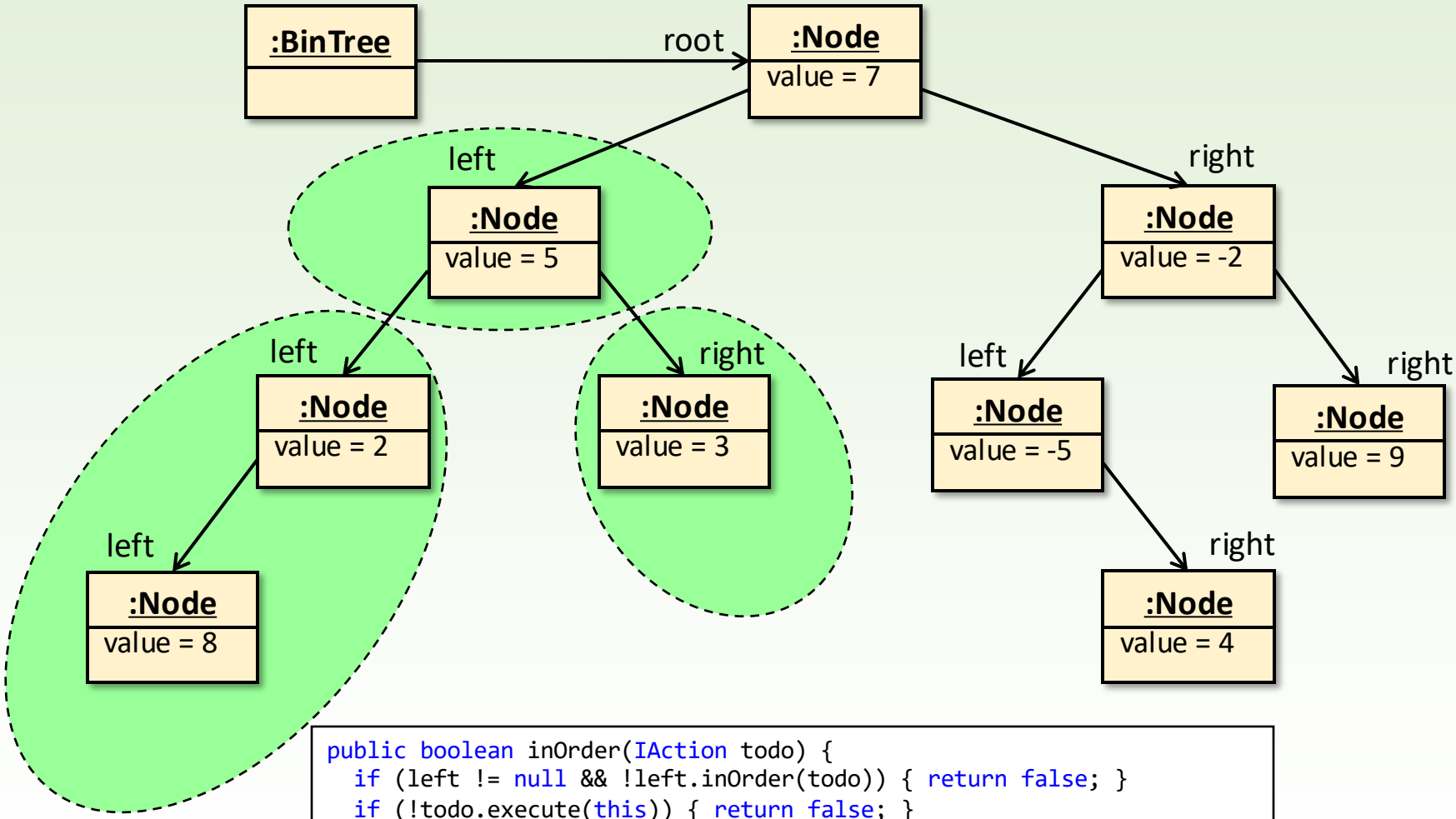
preOrder traversal 7 5 2 8 3 -2 -5 4 9




```
public boolean preOrder(IAction todo) {
  if (!todo.execute(this)) { return false;}
  if (left != null && !left.preOrder(todo)) { return false; }
  if (right != null && !right.preOrder(todo)) { return false; }
  return true;
}
```

inOrder traversal

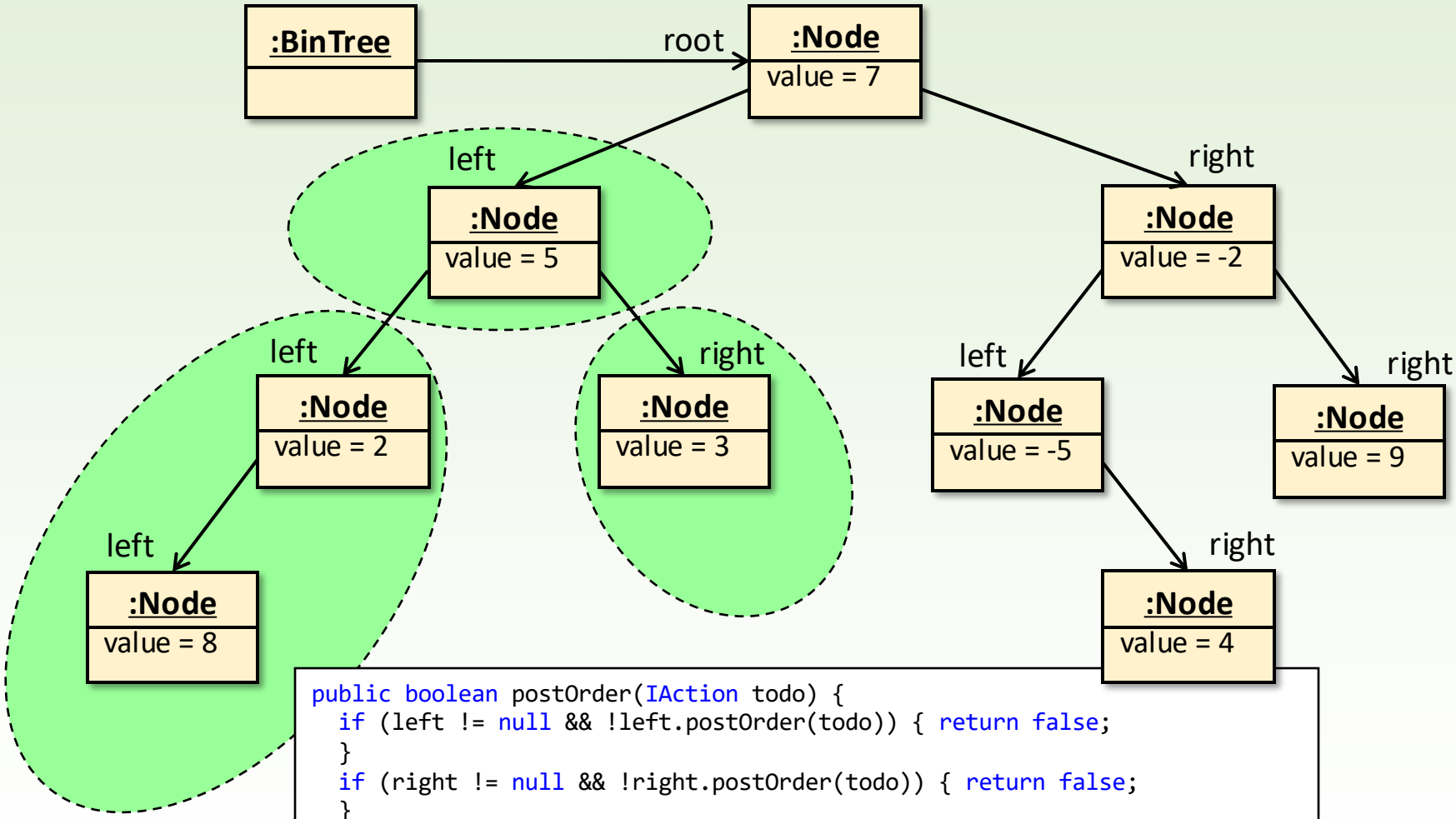
8 2 5 3 7 -5 4 -2 9




```
public boolean inOrder(IAction todo) {
  if (left != null && !left.inOrder(todo)) { return false; }
  if (!todo.execute(this)) { return false; }
  if (right != null && !right.inOrder(todo)) { return false; }
  return true;
}
```

postOrder traversal

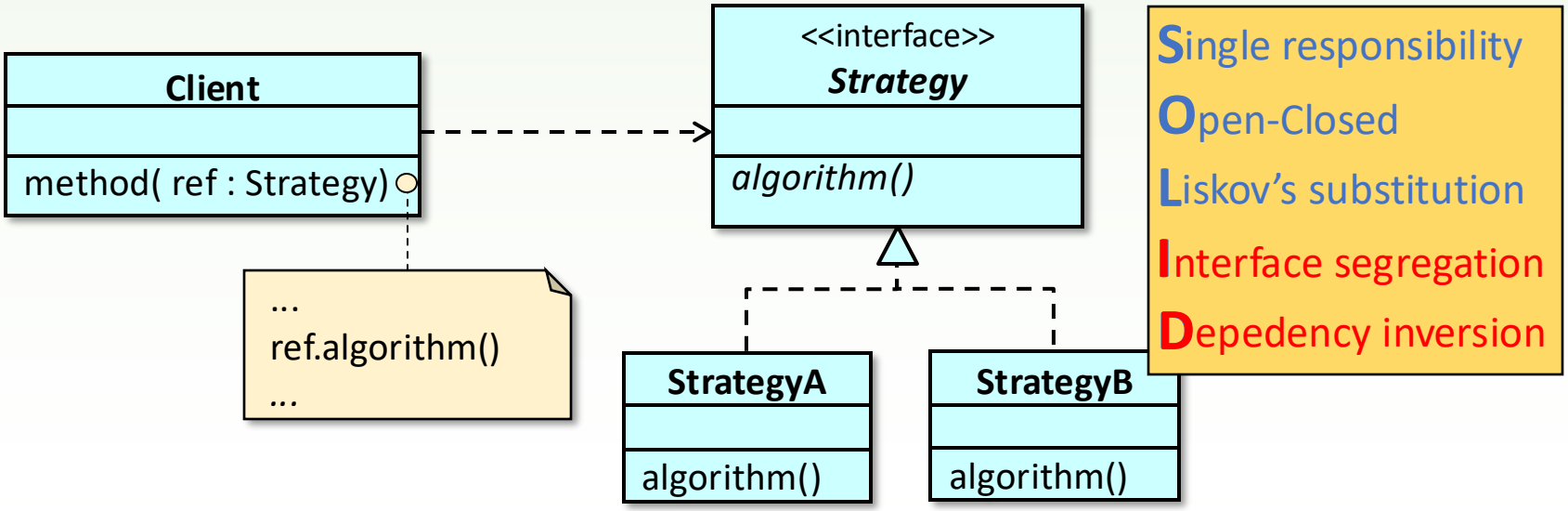
8 2 3 5 4 -5 9 -2 7




```
public boolean postOrder(IAction todo) {
  if (left != null && !left.postOrder(todo)) { return false;
  }
  if (right != null && !right.postOrder(todo)) { return false;
  }
  return todo.execute(this);
}
```

Strategy Design Pattern

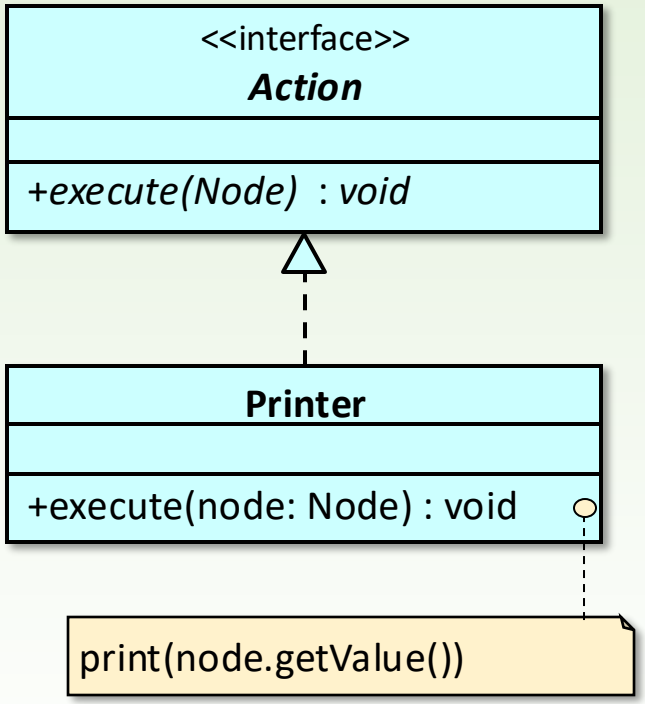
- We introduce a family of activities (algorithms, strategies) as methods of classes that implement a common interface so that it can be determined at runtime which one is used by a method of another (client) class. These activities are carried by individual (Strategy type) objects as their own methods, and this object is received by the client class method as a parameter (or can even be aggregated into the client object).



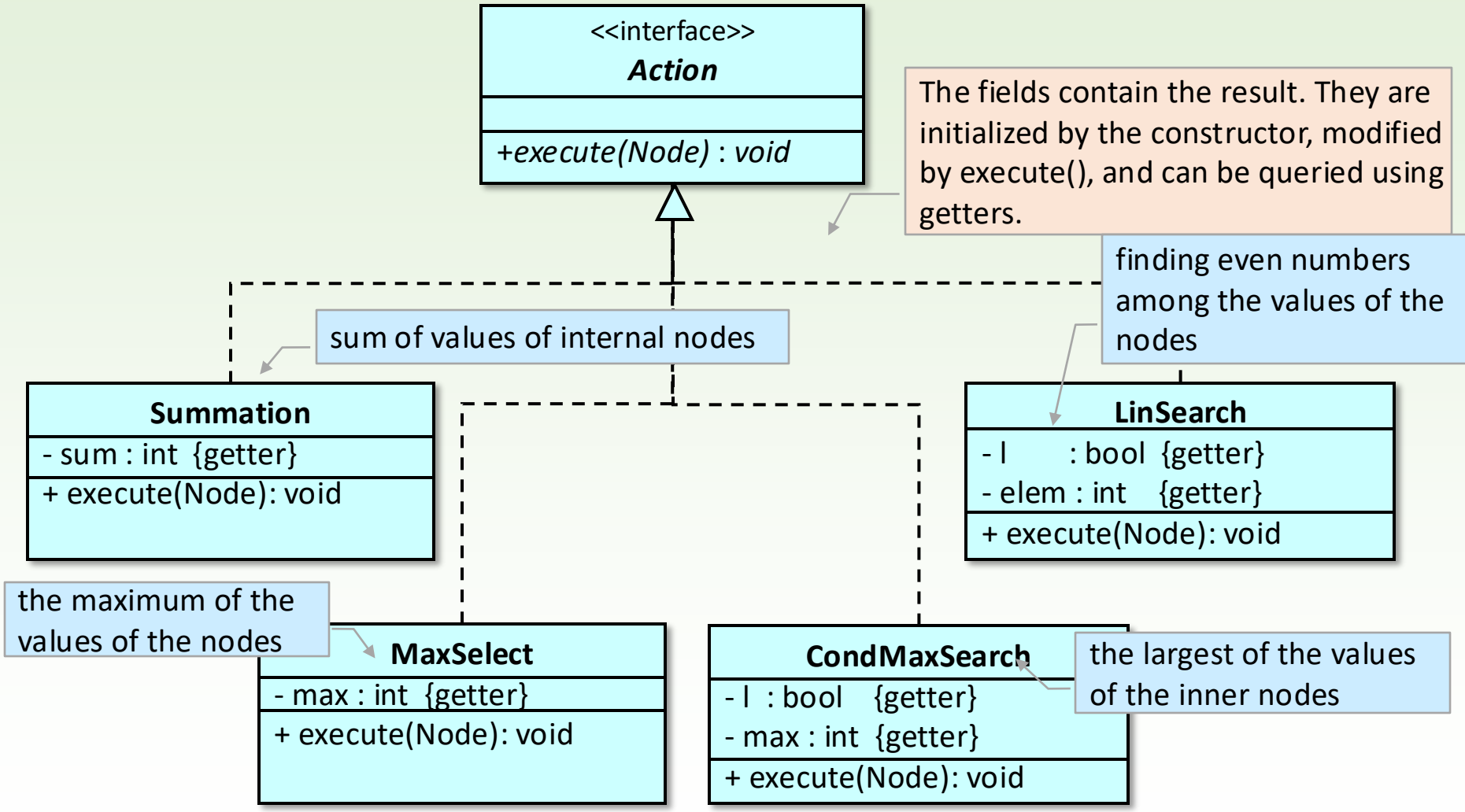
Printer Action class

```
public class Printer implements IAction {
  @Override
  public boolean execute(Node node) {
    System.out.print(" " + node.getValue() + "
");
    return true; // always continue traversal
  }
}
```

```
BinTree t = new BinTree();
 …
Printer print = new Printer();
System.out.print("\nPreorder traversal: ");
t.preOrder(print);

System.out.print("\nInorder traversal: ");
t.inOrder(print);

System.out.print("\nPostorder traversal:");
t.postOrder(print);

System.out.println();
```

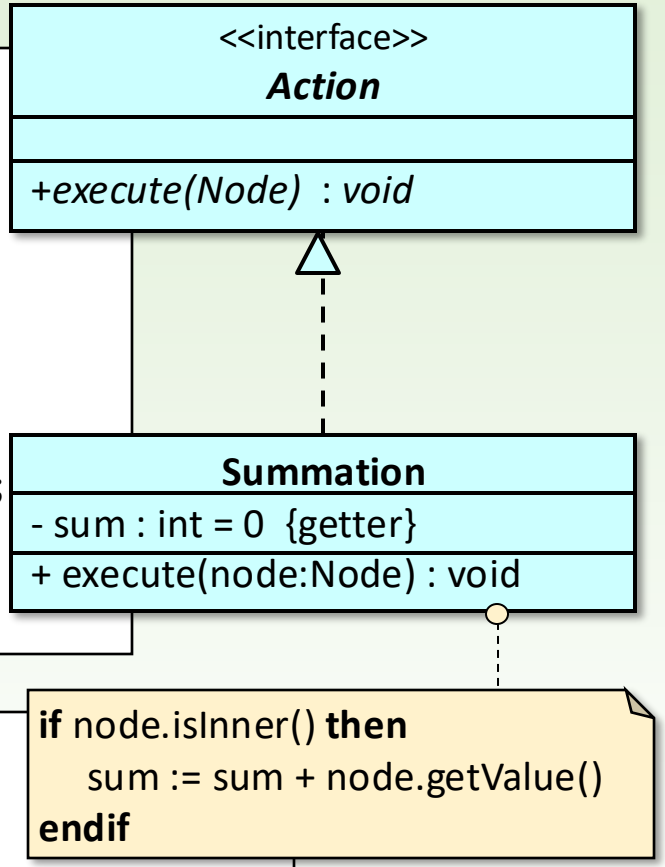


Other Action classes



Action class for summation

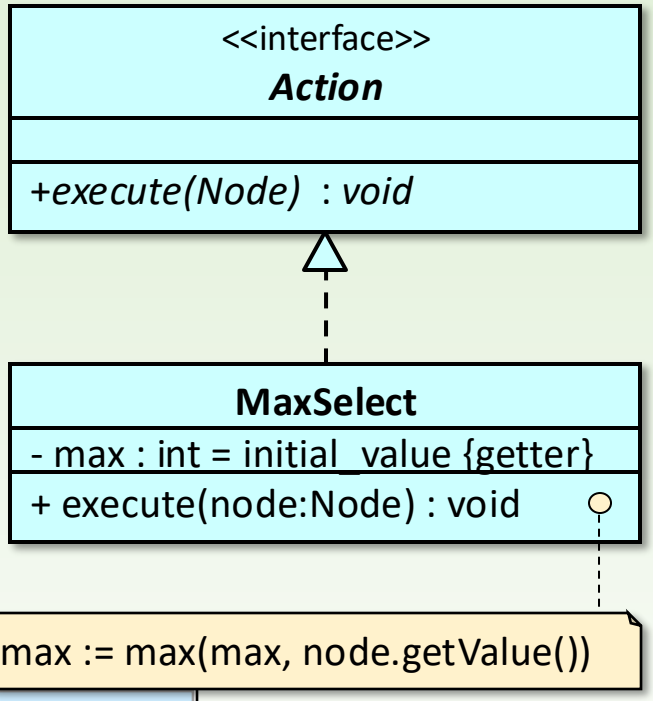
```
public class Summation implements IAction {
    private int sum;

    public Summation() { sum = 0; }

    public int getSum() { return sum; }

    @Override
    public boolean execute(Node node) {
      if (node.isInternal()) sum += node.getValue();
      return true; // always continue traversal
    }
}
```




```
BinTree t = new BinTree();
 …
Summation sum = new Summation();
t.preOrder(sum);
System.out.println("\nSum of internal nodes: " + sum.getSum());
```

Maximum selection

```
public class MaxSelect implements IAction {
    private int max;
    public MaxSelect(int i) { max = i; }
    public int getMax() { return max; }
    @Override
    public boolean execute(Node node) {
        max = Math.max(max, node.getValue());
        return true; // always continue
traversal
    }
}
```



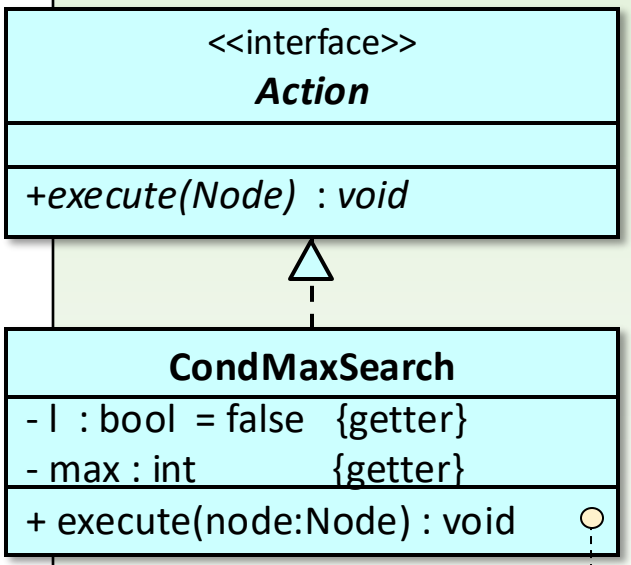

```
BinTree t = new BinTree();
…
 try {
     MaxSelect max1 = new MaxSelect(t.getRoot());
     t.preOrder(max1);
     System.out.println("\nMaximum of elements: " +
max1.getMax());
 } catch (BinTree.NoRootException e) {
     System.out.println("Empty tree.");
 }
```

throws an exception if the binary tree is empty

Conditional maximum search

```
public class CondMaxSearch implements IAction {
  private boolean found;
  private int max;
  public CondMaxSearch() { found = false; }
  public boolean isFound() { return found; }
  public int getMax() { return max; }
  @Override
  public boolean execute(Node node)
  {
    if (node.isInternal())
    {
      if (!found)
      {
        found = true;
        max = node.getValue();
      }
      else { max = Math.max(max, node.getValue()); }
    }
      return true; // always continue traversal
  }
}
```

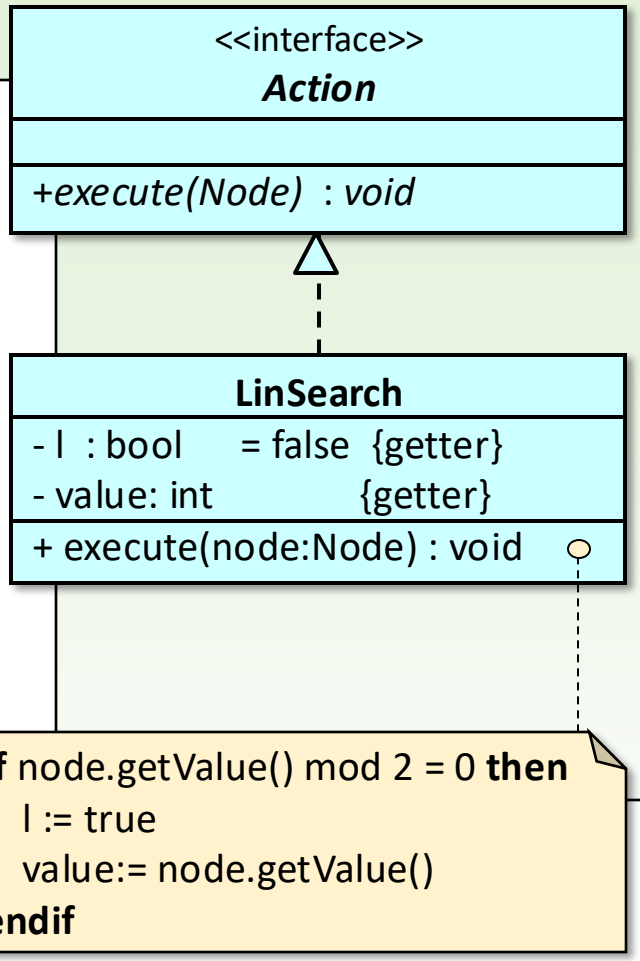
```
BinTree t = new BinTree();
…
CondMaxSearch max2 = new CondMaxSearch();
t.preOrder(max2);
System.out.print("\nMaximum of internal elements:
if (max2.isFound()) System.out.println(max2.getMax
else System.out.println("none");
```



if node.isLeaf() **then return endif**
if not l **then**
    l, max := true, node.getValue()
else max := max(max, node.getValue())
endif

Linear search

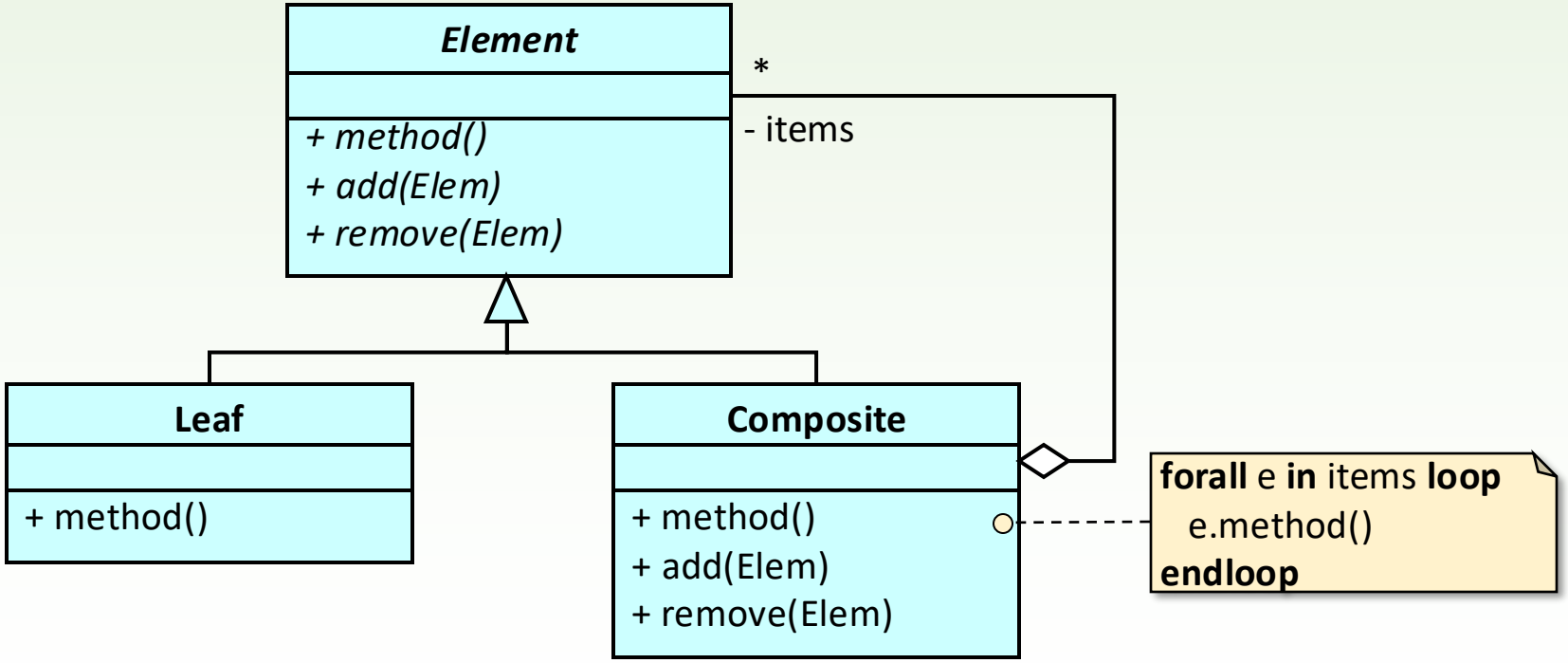
```
public class LinearSearch implements IAction {
    private int value;
    private boolean found;
    public LinearSearch() { found = false; }
    public int getValue() { return value; }
    public boolean isFound() { return found; }
    @Override
    public boolean execute(Node node)
    {
        if (node.getValue() % 2 == 0)
        {
            value = node.getValue();
            found = true;
            return false; // stop traversal
        }
        return true; // continue traversal
    }
}
```

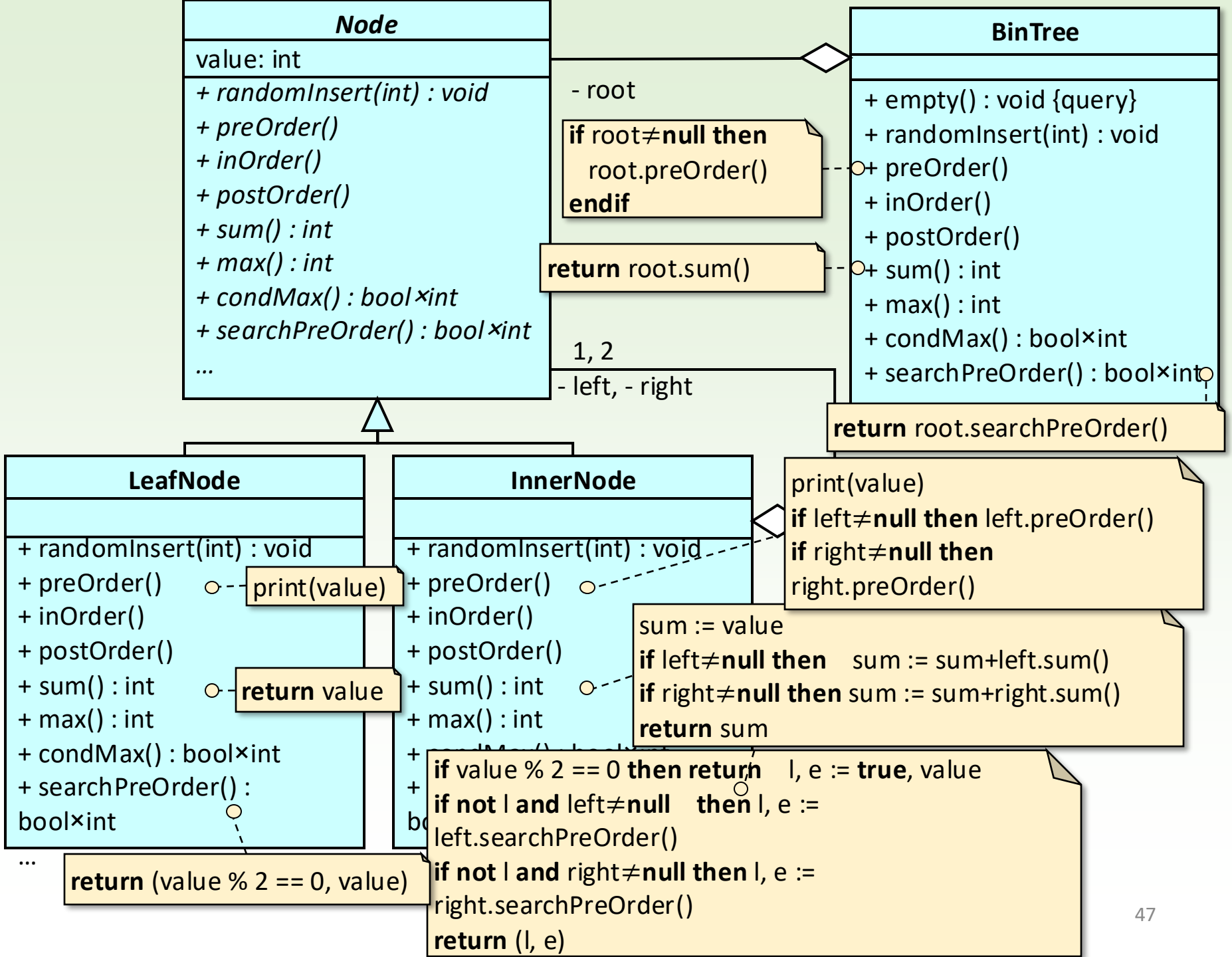



```
BinTree t = new BinTree();
…
LinearSearch search = new LinearSearch();
t.preOrder(search);
if (search.isFound())
  { System.out.println("\nFirst even number: " + search.getValue()); }
Else
  { System.out.println("\nNo even numbers"); }
```

Composite design pattern

- For organizing objects into a tree structure, especially when part-whole (child-parent) relationships need to be managed. Through this method, leaf nodes and subtrees can be handled uniformly.




Node
value: int
+ randomInsert(int) : void
+ preOrder()
+ inOrder()
+ postOrder()
+ sum() : int
+ max() : int
+ condMax() : bool×int
+ searchPreOrder() : bool×int
...
- root
1, 2
- left, - right
BinTree
+ empty() : void {query}
+ randomInsert(int) : void
+ preOrder()
+ inOrder()
+ postOrder()
+ sum() : int
+ max() : int
+ condMax() : bool×int
+ searchPreOrder() : bool×int
if root≠null then
root.preOrder()
endif
return root.sum()
return root.searchPreOrder()
LeafNode
+ randomInsert(int) : void
+ preOrder()
+ inOrder()
+ postOrder()
+ sum() : int
+ max() : int
+ condMax() : bool×int
+ searchPreOrder() : bool×int
...
print(value)
return value
return (value % 2 == 0, value)
InnerNode
+ randomInsert(int) : void
+ preOrder()
+ inOrder()
+ postOrder()
+ sum() : int
+ max() : int
print(value)
if left≠null then left.preOrder()
if right≠null then right.preOrder()
sum := value
if left≠null then sum := sum+left.sum()
if right≠null then sum := sum+right.sum()
return sum
if value % 2 == 0 then return l, e := true, value
if not l and left≠null then l, e := left.searchPreOrder()
if not l and right≠null then l, e := right.searchPreOrder()
return (l, e)